Planners Exploit Deterministic Error: Dithered Low-Bit World Models

Author Names Omitted for Anonymous Review. Paper-ID [add your ID here]

***Abstract*—Sampling-based planners such as MPPI and CEM query a learned world model thousands of times per control step, so low-bit world models are an obvious way to make model-based control cheaper on a robot. With round-to-nearest activation quantization and no calibration, we find that planning can collapse, by up to 100 points of success, while the world model's latent one-step prediction error stays no larger than that of a stochastic rounding scheme that plans well. At the same bit width, what decides the outcome is not the size of the rounding error but whether it persists across planner iterations and is shared across candidates. Round-to-nearest leaves an error that persists, and an iterating planner drifts toward the candidates that this error favors; in TD-MPC2, holding the same dither fixed within a step or an episode brings the drift back. Dither shared by all candidates and redrawn at every iteration removes most of the damage at the same bit width and without training: at least half of it in all 19 affected Meta-World cells of TD-MPC2 (median 95%), in 4 of 5 DINO-WM cells, and in all 8 affected tasks of a 317M-parameter multi-task TD-MPC2. Captured as CUDA graphs on an Ampere GPU, a W8A4 integer TD-MPC2 world model runs its planning rollouts 1.9–3.3× and the whole planning step 1.3–1.5× faster than FP32, though not faster than BF16 with pre-converted weights; shared dither adds at most 2.3% latency. In TD-MPC2, block-scaled NVFP4 removes most of the damage and MXFP4 part of it, which ties the failure to coarse grids with a shared scale. Hypotheses, thresholds and analysis code were fixed before the data that test them, and the hypotheses that failed are reported.**

I. Introduction

A sampling-based planner evaluates its world model candidates × iterations × horizon times per decision: $512 \times 6 \times 3$ forward calls for the MPPI planner of TD-MPC2 [14] and $300 \times 30 \times 5$ for the CEM planner of DINO-WM on PushT [40]. Network evaluations therefore dominate the cost of each control step, and low-bit arithmetic is a direct way to cut it (Sec. VI). Yet quantized world models can break planning in ways that bit width and offline prediction error do not explain [11, 24, 35], unless a calibration objective preserves what planning needs [5]. With round-to-nearest activations and no calibration, we see planning collapse by up to 100 points of success, while in most affected settings the latent one-step error of the world model is no larger than that of a stochastic rounding scheme that plans well.

We show that, at the same bit width, what decides whether a low-bit world model plans well is not the size of its rounding error but whether the error persists across planner iterations and is shared across candidates. Round-to-nearest (det) is a fixed function of its input, so the error it adds to the predicted return of a given action sequence is the same in every planner iteration. Iterating planners are known to drive their candidates to where the model can no longer resolve them [1, 6]; with a persistent error, they keep the candidates that the error favors and move their sampling distribution toward them. Dither, i.e. stochastic rounding $\Delta\lfloor x/\Delta + u \rfloor$ with $u \sim U[0, 1)$, is unbiased; its error is somewhat larger than that of round-to-nearest, but it can be redrawn. Our fix, sh-iter, shares one dither draw across all candidates and redraws it at every forward call, hence at every iteration. Sharing keeps the comparison between candidates fair within an iteration, as with common random numbers [16], and redrawing denies the planner an error it can exploit across iterations.

The ingredients are classical: the optimizer's curse [29], the bias of sample-average approximation [17], common random numbers, and unbiased stochastic rounding for iterative computation [12, 10]. Stochastic rounding is also used at inference in language-model serving, to keep error from accumulating in recurrent state caches [20, 15]. To our knowledge it has not been used to protect sampling-based planning, where its role is different: to deny the planner a persistent error to exploit. What is new is this use inside closed-loop planners, a control that separates sharing from persistence at a fixed per-entry error distribution, and a training-free fix whose effect we measure across two model families, two planners and two model sizes. Our contributions are:

1) **Phenomenon** (Sec. IV). At the same bit width, det causes planning damage that the latent one-step error does not reveal: in 37 of 44 affected cells its error is no larger than that of sh-iter.
2) **Mechanism** (Sec. V). The planner exploits error that persists across its iterations. In TD-MPC2, positive controls that hold the same dither fixed within a step or an episode reproduce 70–130% of det's drift, and the benefit of sh-iter grows with the number of iterations.
3) **Prescription** (Sec. IV). sh-iter removes at least half of the damage in all 19 affected Meta-World cells of TD-MPC2, in 4 of 5 DINO-WM cells and in all 8 affected tasks of a 317M multi-task TD-MPC2, and it is never harmful there.
4) **Deployment** (Sec. VI). Shared dither adds at most 2.3% to the planning rollouts of an integer TD-MPC2 world model and under 1% to the whole planning step, which the integer path makes 1.3–1.5× faster than FP32 but not faster than BF16 with pre-converted weights; in TD-MPC2, block-scaled FP4 formats mark the boundary of the failure.

Every test was fixed in a hashed document before its data, and

the tests that failed are reported (Table I); Sec. VII lists the extensions we propose for Stage 2.

II. Related Work

Quantized world models and policies. Post-training quantization of DINO-WM makes the planning objective and task success diverge at aggressive bit widths [11], and success depends on which modules keep their precision [24]; offline action deviation predicts the closed-loop loss of quantized world action models [35]. A concurrent submission recovers full-precision planning with 4-bit weights through a calibration objective [5]. Quantized vision-language-action and world-action models are benchmarked in closed loop [36, 37, 23] and deployed with fused low-bit kernels and CUDA graphs at about 2× speed-ups [32, 7, 3], and quantization error is known to recur across their denoising loops [3]. These works calibrate, allocate bits or train; none changes the rounding rule or separates how the error is shared from how long it persists. Calibration and dither act on different properties of the error (Sec. VII).

Prediction error versus decisions. Data-averaged prediction error does not track planning quality, while decision-relevant error does [38, 31], and the signed component of rollout error along the goal direction matters more than its magnitude [4]. Min et al. [19] split end-point error across a candidate pool into a pool-common and an action-relative part and repair each with an oracle. Iterating planners drive candidates to where the model cannot resolve them: certified elite decisions become rarer as CEM iterates [1], predicted candidate quality falls to chance at the frame that CEM optimizes [6], and replacing early CEM rounds with a deterministic approximation hurts while score noise of the same size does not [2]. We isolate which property of rounding error the planner exploits, with variants that share one per-entry error distribution, and remove it without a second model.

Exploitation and rounding rules. Planners exploit model error [8]; World Models injected temperature noise into imagined rollouts so that the controller could not exploit them [13]; in CEM the world model also acts as a proposal mechanism [22]. Dither decorrelates quantization error from the signal [25, 28], and unbiased stochastic rounding keeps bias from accumulating [12, 10]. In language models, the best rounding rule differs between layers [9], a deterministic low-discrepancy dither can beat stochastic rounding in recurrent caches [15], and a single fixed stochastic draw of 3-bit weights can be worse than round-to-nearest [34], in line with our positive controls. These results concern error that accumulates within one computation; we study the selection bias that a planner adds when it chooses among candidates.

III. Setup

Models and planners. TD-MPC2 [14]: the official 5M single-task checkpoints (three seeds) on 8 Meta-World tasks [39] (success rate) and 4 DMC tasks [30] (return), planned with the default MPPI [33] (512 samples including 24 policy trajectories, 64 elites, 6 iterations, horizon 3); and the official 317M multi-task checkpoint (mt80; distilled and quantized to FP16 in [18]) on the same 8 Meta-World tasks. DINO-WM [40]: the official PushT, Wall and PointMaze models with open-loop CEM [27] (300 samples, top 30, horizon 5, 30 iterations on PushT and 10 otherwise). All planner settings are the official defaults unless an experiment varies them.

Quantization. We quantize hidden activations: the four hidden layers of the dynamics and reward MLPs in TD-MPC2 (after Mish) and the six MLP outputs of the DINO-WM predictor (after GELU), each to k bits on a uniform grid $[\min, q_{99.99}]$ calibrated per layer on held-out data. Inputs, weights and all other modules stay in floating point except in the integer kernels of Sec. VI. Per task, a rule fixed in advance picks the primary k on held-out episodes as the largest k at which det loses at least 20 points of success (20% of return on DMC), and we test $k-1$, k and $k+1$: 36 cells for TD-MPC2 at 5M and 9 for DINO-WM. For each 317M task we test the primary k (3–5 bits).

Rounding variants. All stochastic variants use the same unbiased dither and therefore the same per-entry error distribution; they differ only in how u is shared and how long it is kept. ind draws u independently per candidate at every forward call. sh-iter shares one draw per layer across all candidates and redraws it at every forward call. sh-step and sh-ep share the same draw but hold it fixed within a control step or an episode (they coincide in open-loop CEM, where we call them sh-plan). det+shift keeps round-to-nearest but shifts the shared grid by a fresh offset at every call.

Metrics and statistics. Every variant is paired with the unquantized model on the same episode seeds and planner noise (90 pairs per cell for TD-MPC2: three checkpoints × 30 seeds, or one 317M checkpoint × 90 seeds; 100 for DINO-WM). The damage D is the drop from that baseline in success points (relative return on DMC). A cell is *affected* if the det drop is real under a one-sided paired bootstrap (10^4 resamples) with Holm correction over the cells of a task. The fraction of damage removed by a variant v is red $= (D_{\mathsf{det}} - D_v)/D_{\mathsf{det}}$, and v is *harmful* if the 95% interval of $D_v - D_{\mathsf{det}}$ lies above 5 points. The latent one-step error is the relative error of the predicted next latent state for the first-iteration candidates at planning time.

Exploitation. Let $e(a)$ be the error of the quantized model's predicted return for candidate a relative to the unquantized model. The optimizer exploitation index OEI $= e(a_{\text{exec}}) - \bar{e}_{\text{first}}$ compares the executed plan with random first-iteration candidates and splits exactly into a *selection* term (the last iteration's pick against its population) and a *drift* term, the change in the population-mean error from the first to the last iteration. Negative drift means the planner moved its population toward candidates that the error favors.

Analysis plan and compute. The study ran in stages, from environment checks to the extensions marked in Table I. Before each stage, its hypotheses, thresholds, seeds, sample sizes and analysis code were frozen in a design document whose hash was logged before any of the stage's data existed; operational

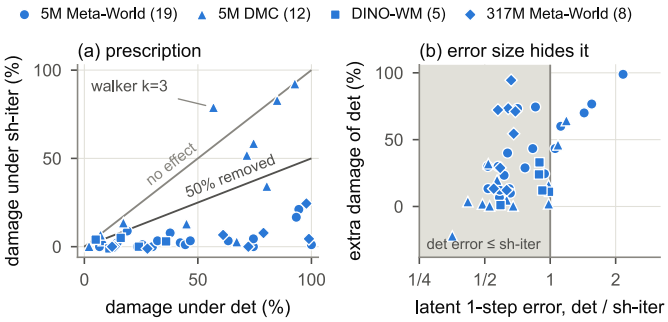


Fig. 1. Every affected cell. (a) Damage under sh-iter vs. det (success points on Meta-World and DINO-WM, relative return on DMC); below the dashed line sh-iter removes at least half of the damage. (b) det's extra damage against the ratio of the two latent one-step errors: most cells sit in the shaded region, where det's error is the smaller one.

choices made in smoke tests were recorded before the main runs. Analyses chosen after seeing data are labeled post hoc. The main study used about 147 GPU-hours on RTX A5000 GPUs, and the extensions used cloud A40, RTX 4090 and RTX 5090 GPUs. Code, design documents and their hashes will be released with the final paper.

IV. Phenomenon and Prescription

Damage that the latent error does not show. det costs up to 100 points (mw-button-press at 2 bits). In 29 of the 36 affected 5M cells its latent one-step error is no larger than that of sh-iter (Fig. 1b). On mw-pick-place at 3 bits, for example, the error is 0.22 for det and 0.39 for sh-iter, yet det loses 37.8 points and sh-iter 7.8. The 317M model shows the same pattern in all 8 affected tasks: det's latent error is 0.55–0.69 times that of sh-iter while det loses 12–99 points. The comparison is within a bit width; across bit widths one-step error and damage do rank together. Two other error measures do separate det: errors over three or five steps (PushT, 4 bits, five steps: 0.25 for det vs. 0.04 for sh-iter), and, observed after the fact, the one-step error of the predicted reward, which is larger under det in all 19 affected Meta-World cells and all 8 tasks at 317M. A deterministic error is correlated across units and adds up in a scalar output, while independent dither errors partly cancel; we have not tested this explanation and list it for Stage 2 (Sec. VII).

Prescription. sh-iter removes at least half of the damage in all 19 affected Meta-World cells (median 95%) and in 4 of 5 DINO-WM cells, and it is not harmful in any of them (Fig. 1a, Table I). The stochastic variants share one per-entry error distribution, yet across the 19 affected Meta-World cells the mean drop is 50.1 points under sh-ep, 34.9 under sh-step, 29.2 under ind and 4.0 under sh-iter, against 44.6 under det; det+shift also loses only 4.0. The weakest Meta-World cell keeps 47% of the damage (window-open, 4 bits), and the DINO-WM exception is PushT at 5 bits, where det's damage is only 5 points. On the 317M multi-task model, which needs no change to the method, sh-iter removes 75–104% of the damage in all 8 tasks (e.g. mw-peg-insert-side: 98.9 points lost under det, 4.4 under sh-iter), and the 95% interval of $D_{\text{sh-iter}} - D_{\text{det}}$ lies below zero in every task. The primary bit widths at 317M (3–5) are close to those at 5M (3–4), so the larger model is not more robust to rounding.

Where it helps less. On DMC, sh-iter removes half of the damage in only 5 of 12 cells and is harmful on walker at 3 bits (78.9% of return lost vs. 56.9% under det). A rule added after the first stage and tested on new seeds explains where it helps: the persistent share of det's favorable error (drift over OEI) is 0.33–0.52 on DMC and 0.54–0.73 on Meta-World, and across the 12 TD-MPC2 tasks it predicts the damage removed (Spearman 0.83, new seeds). The two DMC tasks with the smallest persistent share are the two where sh-iter fails.

V. Mechanism

Drift. Under det the population-mean error drifts in the favorable direction in all four mechanism tasks at their primary bit widths. sh-iter keeps at most 24% of that drift (0.10, 0.24, 0.03 and -0.01 of it), and det+shift keeps at most 15%. The positive controls isolate persistence: holding the *same* shared dither fixed for a control step (sh-step) or an episode (sh-ep) brings back 70–130% of det's drift in the three TD-MPC2 tasks (Fig. 2a). Error size and sharing are equal across these variants; only persistence differs. On PushT the full-size fixed table did not reproduce the drift (ratio 0.01); there a fixed dither was exploited only when we made its table of offsets coarse, as below.

Iterations, candidates and table size. The fix needs an iterating planner. With one MPPI iteration sh-iter is no better than det; at 12 iterations its damage is 0–11% while det's stays at 43–47% and sh-step's at 18–47% (Fig. 2b). With the number of elites fixed, det's exploitation $|\text{OEI}|$ and damage grow with the number of candidates in three of four tasks, the dose-response expected from the optimizer's curse [29]. A fixed dither drawn from a table of N_{eff} independent offsets hurts less as the table grows (five of six tasks, three of them exactly at the threshold), and in an exploratory Wall cell (3 bits) a single offset per layer collapsed planning (+37 points).

Theory. For a planner that averages fresh unbiased noise, the expected drift is zero. A persistent error field instead pulls the population toward the minimizer of the smoothed model cost, the bias of sample-average approximation [17]. The selection and drift terms we derived matched 922k simulated MPPI and CEM runs (fitted slopes 0.92–1.05) and predicted, before the first stage was analyzed, that the OEI would be negative even under fresh noise, because the last iteration's pick benefits from the noise it was selected on (the winner's curse). This is why our original OEI tests, H2a and H2b, failed, and why our mechanism claim concerns the drift rather than the OEI.

VI. Deployment: Latency and Number Formats

Integer kernels. A fused Triton kernel (int8 weights, 4- or 8-bit activation codes, shared dither drawn on the GPU) reproduces the fake-quantization results in closed loop, and 8-bit activations cause no damage. Run eagerly at 5M parameters

TABLE I

HYPOTHESES AND OUTCOMES. EACH TEST WAS FIXED BEFORE ITS DATA; ROWS MARKED † WERE ADDED AFTER THE FIRST STAGE AND TESTED ON NEW SEEDS; ‡ MARKS THE EXTENSION EXPERIMENTS RUN FOR THIS SUBMISSION. MECHANISM TASKS: MW-PUSH, MW-PICK-PLACE, CHEETAH-RUN, PUSHT.

	Hypothesis and test	Outcome
H1	det's latent one-step error $\leq$ sh-iter's in some affected cell of each family	holds: 29/36 cells (TD-MPC2 5M, DINO-WM); 317M‡ 8/8
H3a	sh-iter removes $\geq$50% of the damage in $\geq$75% of affected cells, none harmful	Meta-World 19/19 (median 95%), DINO-WM 4/5, 317M‡ 8/8; DMC fails (5/12)
DMC†	Spearman(persistent share of det's error, red of sh-iter) ≥ 0.5 over 12 tasks	holds: 0.83
H2a′†	det drift < 0 and drift(sh-iter)/drift(det) < 0.5	holds: 4/4 tasks (ratio ≤ 0.24)
H2c	Spearman(candidate count, $\lvert\text{OEI}_{\text{det}}\rvert$ and D_{det}) ≥ 0.8, elites fixed	holds: 3/4 tasks
H4a	Spearman(table size of fixed dither, its damage) ≤ -0.8	holds: 5/6 tasks (three exactly at -0.8)
H3b	sh-iter beats det on top of the best post-training quantization	TD-MPC2 holds (10/19); DINO-WM fails (1/3)
H5	Dither latency overhead $\leq$10% in a fused integer kernel	holds (max 6.9%); integer kernel 18% slower than FP32 (eager)
L1–L3‡	Integer rollout $\leq 0.8\times$ FP32 time and $\leq 0.9\times$ the faster of TF32/BF16 (autocast) in some cell; dither $\leq$5% everywhere	all hold: 12/12, 10/12, max overhead 2.1%; replicated in a second run (2.3%)
L2c‡	As L2, with pre-converted BF16 among the float formats	fails: 0/12 (1.07–1.52$\times$ its time)
S1–S3‡	As L1–L3 (with pre-converted BF16) for the whole planning step	S1, S3 hold (12/12; max 0.7%); S2 fails (0/12, 1.01–1.12$\times$)
F1–F3‡	FP4 formats: det damages $\geq$3 of 12 tasks; sh-iter removes $\geq$50%; det's error $\leq$ sh-iter's	NVFP4 fails F1 (2/12), MXFP4 holds (4/12); F2 holds on Meta-World (1 cell), fails on DMC (2/5); F3 holds (6/6)
H2a, H2b	Original OEI tests: OEI$\approx$0 under sh-iter; OEI predicts damage better than one-step error	fail (claims narrowed to drift)
H4b	Benefit of sharing grows with the density of near-ties	fails ($\rho = 0.03$)

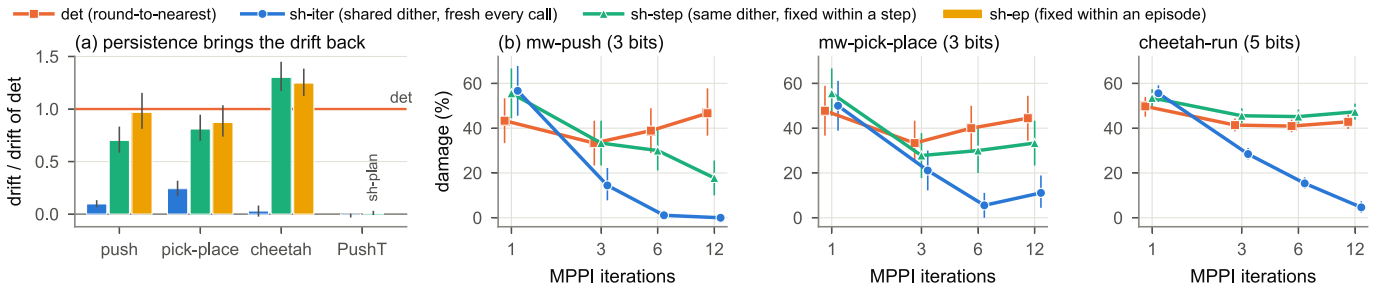


Fig. 2. Mechanism. (a) Favorable drift of the population-mean error kept by each variant, relative to det (TD-MPC2 at its primary bit width; PushT at 4 bits, where sh-step and sh-ep coincide as sh-plan); bars show 95% bootstrap intervals. (b) Damage vs. MPPI iterations (TD-MPC2, 90 paired episodes per point, 95% intervals). Fresh dither needs iterations to help; det does not improve with them.

and 512 candidates, it was 18% slower than FP32: the step is bound by kernel launches, not arithmetic. To map where low bit widths pay off, we timed the world-model rollouts of one control step (dynamics and reward for N candidates, plus the policy trajectories) replayed as a single CUDA graph, as TD-MPC2's compiled planner does. The integer path uses int8 weights (per output channel) and 4-bit activation codes with int8 tensor-core GEMMs; the first layer, whose input $[z, a]$ is not quantized, stays in FP32. On an NVIDIA A40, across 5M–317M parameters and $N \in \{512, 2048, 8192\}$ (10 interleaved rounds per condition), the integer rollout is 1.9–3.3$\times$ faster than FP32 in all 12 cells (Fig. 3a); shared dither changes latency by -0.2 to $+2.1$%, because one draw per layer and call serves every candidate. The gains are largest at 317M, the size at which the multi-task results of Sec. IV hold. A second run on another A40 reproduced these speed-ups (1.9–3.3$\times$; dither at most $+2.3$%) and tested a stronger BF16 baseline, with the weights converted once in advance instead of at every call (autocast): it is then the fastest float format in all 12 cells, and our integer rollout takes 1.07–1.52$\times$ its time. Much of the gap is the first layer, which we keep in FP32; with TF32 there, the integer rollout takes 0.72–0.96$\times$ the time of pre-converted BF16 (post hoc), but we have not evaluated that path in closed loop. Because the dynamics and reward networks are only 40–52% of an FP32 planning step, the whole step (encoder, policy, Q-ensemble and MPPI captured as one CUDA graph) gains less: 1.28–1.49$\times$ over FP32 (Fig. 3b), while taking 1.01–1.12$\times$ the time of the step with pre-converted BF16; shared dither adds at most 0.7% to it.

FP4 formats mark the boundary. On an RTX 5090, an NVFP4 GEMM at the 317M layer width is up to 5.3$\times$ faster than BF16, so we repeated the 5M experiment with activations in NVFP4 [21] (blocks of 16, E4M3 scale) and MXFP4 [26] (blocks of 32, power-of-two scale), rounding each block to the E2M1 grid to nearest (det) or stochastically (sh-iter). det damaged only 2 of 12 tasks under NVFP4 (cheetah-run 9.9%, walker 1.7%), short of the three our analysis plan required, and 4 of 12 under MXFP4 (cheetah-run 31.3%, window-open 10.0 points), against 71.6% on cheetah-run with a uniform 4-bit grid. Where damage remained, the signature was the same:

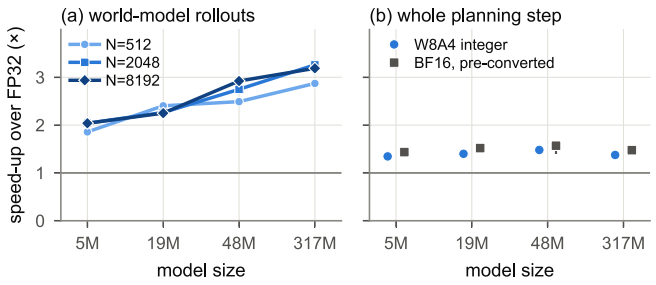


Fig. 3. Speed-up over FP32 on an NVIDIA A40, as CUDA graphs. (a) W8A4 world-model rollouts of one MPPI step (N candidates, 6 iterations, horizon 3). (b) Whole planning step, W8A4 and BF16 with pre-converted weights: median over N, bars show the range. (a) and (b) are two runs on different A40s.

det's latent one-step error was 0.48–0.63 times that of sh-iter in all six damaged cells, and sh-iter removed most of the damage in the one damaged Meta-World cell but in only 2 of 5 DMC cells, as with uniform grids. Block scaling shrinks the error and dither removes its persistence, two different axes; in TD-MPC2 the failure we study belongs to coarse grids with a shared scale, while quantized VLA policies can still lose success under NVFP4 [23].

VII. Limitations and Stage 2 Plan

Limitations. The prescription assumes an iterating planner, and on DMC it helps in fewer than half of the affected cells. On top of the best post-training quantization (mostly per-channel ranges), which already removes most of det's damage, the extra gain of sh-iter is borderline in TD-MPC2 (10 of 19 cells) and absent in DINO-WM (1 of 3). We did not combine dither with calibration objectives such as [5]. Our quantization-aware-training baseline was weak, so we draw no conclusion about training. The 317M results come from one checkpoint and the FP4 results from fake quantization of the 5M models. The latency map comes from one GPU type, on which BF16 with pre-converted weights, not evaluated in closed loop, is faster than our integer path with its first layer in FP32. We have no physical-robot experiments.

Proposed Extension Charter. As in Table I, each item would have its hypotheses, thresholds and analysis code fixed before its data.

1) *A diagnostic that sees the damage.* Unlike the latent one-step error, the one-step reward error separated det in every affected Meta-World cell (post hoc). We will test on new tasks and seeds whether the correlation of rounding error across hidden units predicts det's damage within a bit width, and whether a reward-error check before deployment flags the damaging configurations.
2) *Integer kernels for DINO-WM.* We will port the integer path to the DINO-WM predictor and test whether closed-loop damage and repair match fake quantization in the affected cells, with the same latency budget for dither ($\leq$5%).
3) *Closing the gap to BF16.* We will move the integer path's first layer out of FP32 and evaluate that path and BF16 with pre-converted weights in closed loop, comparing success and latency on the same planner.
4) *Embedded and FP4 hardware.* We will repeat the latency map, against BF16 with pre-converted weights, on an NVIDIA Jetson module and, with fused quantization kernels, for NVFP4 on a Blackwell GPU, including the cost of drawing dither.

The negative results above (DMC, the combination with strong post-training quantization, planners with few iterations) stay in scope as boundaries to report, not as targets to tune away.

References

[1] Anonymous. When Can a World Model Decide? Certifying Elite Decisions in Sampling-Based Planning. Submitted to ICLR 2027, 2026. OpenReview cVCKr4bBU2.

[2] Anonymous. Not All Evaluations Are Equal: Where Test-Time Compute Matters in World-Model Planning. Submitted to ICLR 2027, 2026. OpenReview 1vmM17t1R8.

[3] Anonymous. LoopQuant: Loop-Aware Post-Training Quantization for Vision-Language-Action Models. Submitted to ICLR 2027, 2026. OpenReview AhG6FcJBHt.

[4] Anonymous. What Rollout Error Magnitude Misses in Latent World-Model Planning. Submitted to ICLR 2027, 2026. OpenReview 5alhlhb7zd.

[5] Anonymous. What Must Quantization Preserve in Latent World Models? Submitted to ICLR 2027, 2026. OpenReview 3VSa1Kx7HD.

[6] Anonymous. Resolution Collapse in Latent-Space CEM Planning. Submitted to ICLR 2027, 2026. OpenReview eTu4mzKor3.

[7] Anonymous. WAMPTQ: Beyond Weight Quantization via Task-Utility-Guided Precision Allocation for Efficient World Model Deployment. Submitted to ICLR 2027, 2026. OpenReview hdBfu8lIC7.

[8] Rinu Boney, Norman Di Palo, Mathias Berglund, Alexander Ilin, Juho Kannala, Antti Rasmus, and Harri Valpola. Regularizing trajectory optimization with denoising autoencoders. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[9] Yohan Chatelain and Pablo de Oliveira Castro. Stochastic Rounding in Low-Precision Transformer Inference: A Variable-Precision Emulation Study of a Small GPT-2. *arXiv preprint arXiv:2610.01889*, 2026.

[10] Matteo Croci, Massimiliano Fasi, Nicholas J. Higham, Theo Mary, and Mantas Mikaitis. Stochastic rounding: Implementation, error analysis and applications. *Royal Society Open Science*, 9(3):211631, 2022.

[11] Zhongqian Fu, Tianyi Zhao, Kai Han, Hang Zhou, Xinghao Chen, and Yunhe Wang. An Empirical Study of World Model Quantization. *arXiv preprint arXiv:2602.02110*, 2026.

[12] Suyog Gupta, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. Deep learning with limited

numerical precision. In *International Conference on Machine Learning (ICML)*, pages 1737–1746, 2015.

[13] David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[14] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations (ICLR)*, 2024.

[15] Snigdha Chandan Khilar. Low-Discrepancy Dither for Quantized Recurrent State Caches. *arXiv preprint arXiv:2609.39185*, 2026.

[16] Nathan L. Kleinman, James C. Spall, and Daniel Q. Naiman. Simulation-based optimization with stochastic approximation using common random numbers. *Management Science*, 45(11):1570–1578, 1999.

[17] Anton J. Kleywegt, Alexander Shapiro, and Tito Homem-de Mello. The sample average approximation method for stochastic discrete optimization. *SIAM Journal on Optimization*, 12(2):479–502, 2002.

[18] Dmytro Kuzmenko and Nadiya Shvai. TD-MPC-Opt: Distilling Model-Based Multi-Task Reinforcement Learning Agents. *arXiv preprint arXiv:2507.01823*, 2025.

[19] Rui Min, Xianyao Li, Fang Xu, and Jing Du. When Low Prediction Error Misleads Planning: Diagnosing Representation, Dynamics, and Decision Failures in Latent World Models. *arXiv preprint arXiv:2610.05550*, 2026.

[20] NVIDIA. Nemotron 3 Super: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning. *arXiv preprint arXiv:2604.12374*, 2026.

[21] NVIDIA, Felix Abecassis, Anjulie Agrusa, et al. Pretraining large language models with NVFP4. *arXiv preprint arXiv:2509.25149*, 2025.

[22] Oliver Obst and Frieder Stolzenburg. In CEM, a World Model Is Also a Proposal Mechanism. *arXiv preprint arXiv:2610.00921*, 2026.

[23] Seongmin Park, Jaewon Lee, Yujun Heo, et al. Recovering Low-Bit Failures in Vision-Language-Action Policies via Quantization-Conditioned Reinforcement Learning. *arXiv preprint arXiv:2610.15675*, 2026.

[24] Suraj Ranganath, Anish Patnaik, and Vaishak Menon. Where Bits Matter in World Model Planning: A Paired Mixed-Bit Study for Efficient Spatial Reasoning. *arXiv preprint arXiv:2602.11882*, 2026.

[25] Lawrence G. Roberts. Picture coding using pseudo-random noise. *IRE Transactions on Information Theory*, 8(2):145–154, 1962.

[26] Bita Darvish Rouhani, Ritchie Zhao, Ankit More, et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.

[27] Reuven Y. Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190, 1999.

[28] Leonard Schuchman. Dither signals and their effect on quantization noise. *IEEE Transactions on Communication Technology*, 12(4):162–165, 1964.

[29] James E. Smith and Robert L. Winkler. The optimizer's curse: Skepticism and postdecision surprise in decision analysis. *Management Science*, 52(3):311–322, 2006.

[30] Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, Timothy Lillicrap, and Martin Riedmiller. DeepMind control suite. *arXiv preprint arXiv:1801.00690*, 2018.

[31] Linhao Wang, Yiyan Fan, and Dongjin Huang. Not All Errors Matter: Decision-Relevant Prediction Error Predicts Planning Quality. *arXiv preprint arXiv:2609.32322*, 2026. v2.

[32] Yunhan Wang, Haodong Wang, Zhiming Liu, et al. SteerQuant: Steering Quantization Error with Action-Guided Scaling in World-Action Models. *arXiv preprint arXiv:2609.39056*, 2026.

[33] Grady Williams, Nolan Wagener, Brian Goldfain, Paul Drews, James M. Rehg, Byron Boots, and Evangelos A. Theodorou. Information theoretic MPC for model-based reinforcement learning. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 1714–1721, 2017.

[34] Zekun Wu, Swati Dhiman, and Adriano Koshiyama. Which Decisions Low-Bit Quantization Breaks, and How to Predict Them. *arXiv preprint arXiv:2608.06564*, 2026.

[35] Jiuyi Xu, Jinjia Guo, Meida Chen, Jing Du, and Yangming Shi. Predict Before You Deploy: Offline Prediction of Quantization-Induced Task Degradation for World Action Models. *arXiv preprint arXiv:2609.19441*, 2026.

[36] Jiuyi Xu, Qing Jin, Meida Chen, Song Wang, Yang Sui, and Yangming Shi. VLAQuantBench: Closed-Loop Evaluation of Post-Training Quantization for Vision-Language-Action Models. *arXiv preprint arXiv:2609.25376*, 2026.

[37] Yuhao Xu, Yantai Yang, Zhenyang Fan, et al. QVLA: Not All Channels Are Equal in Vision-Language-Action Model's Quantization. In *International Conference on Learning Representations (ICLR)*, 2026.

[38] Hanzhe You, Yonggang Zhang, Maohao Ran, et al. A Control Theory of Predictability in Latent World Models. *arXiv preprint arXiv:2607.10362*, 2026.

[39] Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Conference on Robot Learning (CoRL)*, 2019.

[40] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024.